AIRA

Academic Integrity Risk Analyzer

Reinforcement Learning for Agentic Policy Audit Systems

Technical Report -- Take-Home Final

INFO7375: Reinforcement Learning for Agentic AI Systems

Northeastern University, Spring 2025

Student: Rajesh Ramareddy

Email: ramareddy.r@northeastern.edu

GitHub: github.com/rajesh1997r/AIRA-policy-auditor

Abstract

AIRA is a budget-constrained reinforcement learning system that audits academic policy documents for contradictory clause pairs. A Deep Q-Network (DQN) agent paired with Upper Confidence Bound (UCB1) exploration learns to identify which clause pairs warrant comparison within a 15% budget constraint. Trained on a 17-clause synthetic policy using real sentence-transformer embeddings (all-MiniLM-L6-v2), AIRA achieves F1=0.827 on held-out seeds and Recall=0.981 while reviewing only 8.1% of all clause pairs. Zero-shot evaluation on Northeastern and MIT AI policies yields F1=0.179, revealing that generalisation requires training on multi-document corpora. The system is structured as a multi-agent pipeline (SegmenterAgent, EmbedderAgent, RLAuditAgent, ReportAgent) coordinated by an AuditController with cross-document ContradictionMemory. All code, data, trained models, and results are openly available.

1. Introduction and Problem Statement

University AI usage policies frequently contain contradictory clauses that create ambiguous and inconsistent expectations for students and faculty. For example, one clause may broadly permit AI writing assistants while another prohibits them entirely. Manual review of these policies is subjective, slow, and error-prone -- especially as policies grow in length and complexity.

AIRA (Academic Integrity Risk Analyzer) addresses this with a reinforcement learning agent that learns to efficiently detect contradictory clause pairs under a strict budget constraint: the agent may review at most 15% of all possible clause pairs per audit episode. This budget constraint makes the problem non-trivial and unsuitable for simple keyword matching -- the agent must learn which semantic features signal contradiction risk.

Why RL over rule-based NLP? Contradiction detection is context-dependent: the same vocabulary patterns (permit / prohibit) may or may not constitute a contradiction depending on scope, subject, and authority. An RL agent can learn nuanced decision boundaries from reward signals that reward true discoveries and penalise missed contradictions -- objectives that are difficult to encode as deterministic rules.

RL Methods Implemented

* Value-Based Learning (DQN): learns Q(s, a) over clause-pair states
* Exploration Strategies (UCB1): adds exploration bonus to under-reviewed pairs

Agentic System Type

* Research / Analysis Agent: automated policy quality assurance
* AuditController orchestration layer with 4 specialised sub-agents
* ContradictionMemory for cross-document pattern accumulation

2. System Architecture

The AIRA pipeline consists of four specialised agents coordinated by AuditController:

```
Policy Text (.txt)
      |
      v
SegmenterAgent  -->  Clause List [{id, text, section, source_doc}]
      |
      v
EmbedderAgent   -->  Embeddings (N x 384, L2-normalised, all-MiniLM-L6-v2)
      |
      v
ClauseAuditorEnv (Gymnasium MDP)
 State : [emb_A(384) | emb_B(384) | cosine_sim | len_ratio |
           budget_used_frac | step_frac]  =  772-dim
 Action: 0=SKIP, 1=COMPARE
 Budget: 15% of all N*(N-1)/2 pairs per episode
      |
      v
RLAuditAgent (DQN + UCB1)  -->  Flagged contradiction pairs
      |
```

```
        v
ReportAgent  -->  AuditReport {precision, recall, F1, efficiency, FlaggedPairs}
        |
        v
ContradictionMemory  -->  Cross-document pattern detection
```

Sub-Agent Roles

Sub-Agent	Responsibility
SegmenterAgent	Splits raw policy text into discrete clauses (>15 tokens, section-aware)
EmbedderAgent	Maps clauses to 384-dim semantic vectors; caches to disk for efficiency
RLAuditAgent	Runs DQN+UCB episode; returns FlaggedPair list with confidence scores
ReportAgent	Computes metrics (Precision/Recall/F1/Efficiency) and formats AuditReport
ContradictionMemory	Stores per-document TP sets; surfaces types found in multiple docs

3. MDP Formulation

AIRA is formalised as a finite-horizon Markov Decision Process M = (S, A, R, T, gamma).

3.1 State Space

For a document with N clauses, each episode presents one clause pair (c_i, c_j) at a time. The state vector is:

```
S_t in R^772 =
  [ phi(c_i)  (384 dims) ]   -- sentence embedding of clause A
  [ phi(c_j)  (384 dims) ]   -- sentence embedding of clause B
  [ cos(phi_i, phi_j)  (1) ] -- cosine similarity (dot product, normalised)
  [ lambda(c_i, c_j)   (1) ] -- |len_A - len_B| / max(len_A, len_B)
  [ b_t                (1) ] -- budget fraction used so far
  [ tau_t              (1) ] -- step fraction (step / total_pairs)
Total: 384 + 384 + 1 + 1 + 1 + 1 = 772
```

phi is the all-MiniLM-L6-v2 sentence transformer (L2-normalised, 384-dim).

3.2 Action Space

A = {0 = SKIP, 1 = COMPARE} (Discrete(2))

COMPARE flags the pair as a potential contradiction and consumes one budget unit. SKIP passes without review.

3.3 Reward Function

```
R(a, is_contradiction) =
  +2.0  if a == COMPARE and is_contradiction       (True Positive)
  -0.1  if a == COMPARE and not is_contradiction   (False Positive)
  -0.5  if a == SKIP   and is_contradiction        (False Negative / Miss)
   0.0  if a == SKIP   and not is_contradiction    (True Negative)
```

Design rationale: The miss penalty (-0.5) is 5x larger than the false-positive penalty (-0.1), driving the agent towards high recall. The asymmetry means the agent learns to err on the side of caution -- flag ambiguous pairs -- rather than skip them. The +2.0 true-positive reward provides a strong incentive to seek out genuine contradictions. If the budget is exhausted before all pairs are reviewed, all unreviewed ground-truth contradictions incur an additional -0.5 penalty each.

3.4 Termination and Discount

```
Terminated = (budget_used >= floor(0.15 * N_pairs)) OR (all pairs reviewed)
gamma = 0.99  (finite-horizon; keeps distant budget-penalty terms from dominating)
```

3.5 Curriculum Learning

The budget fraction linearly decays from 0.30 (episode 0) to 0.15 (episode 200), then stays fixed:

```
budget(ep) = 0.30 - min(ep / 200, 1.0) * (0.30 - 0.15)
```

A wider initial budget ensures the replay buffer is populated with positive (COMPARE) transitions early in training, preventing the agent from learning a trivial all-SKIP policy.

4. Reinforcement Learning Methods

4.1 Deep Q-Network (DQN) -- Value-Based Learning

The Q-network maps the 772-dim state to Q-values for both actions:

```
Architecture: 772 -> 512 (BN + ReLU) -> 256 (BN + ReLU) -> 128 (ReLU) -> 2
Optimizer   : Adam (lr = 1e-4)
Loss        : Huber (SmoothL1) -- robust to asymmetric reward outliers
Batch size  : 64
Buffer      : 50,000 transitions (random experience replay)
Target net  : hard copy every 100 gradient steps
Epsilon     : linear decay 1.0 -> 0.05 over 10,000 gradient steps
Grad clip   : max_norm = 10.0
```

Bellman update:

```
Q(s,a) <- r + gamma * max_{a'} Q_theta_minus(s', a')
Loss   = SmoothL1( Q_theta(s,a),  r + gamma * max_{a'} Q_theta_minus(s', a') )
```

BatchNorm1d on hidden layers requires explicit train()/eval() mode switching during inference. The target network always stays in eval() mode; the online network switches to eval() for single-sample Q-value queries and back to train() for gradient updates.

4.2 UCB1 Exploration Strategy

UCBExplorer wraps DQNAgent and augments the COMPARE action Q-value with an exploration bonus:

```
score(i,j) = Q_DQN(s_{ij}, COMPARE)  +  c * sqrt( ln(t + 1) / N(i,j) )

c      = 0.3   (tuned; see ablation study in Section 6.3)
t      = total pair-selection steps accumulated across ALL episodes
N(i,j) = visit count for canonical pair (i,j) -- also accumulated across episodes

pair_to_idx(i, j, n) = i*n - i*(i+1)//2 + (j-i-1)   [triangular mapping]
```

The UCB bonus applies ONLY to the COMPARE action. Semantically, this is correct: UCB encourages examining unseen or rarely-reviewed pairs before deciding to skip them. Visit counts accumulate ACROSS episodes (not reset each episode) -- this is the key design choice that produces the exploration-to-exploitation shift observed in training.

Important: UCB is a TRAINING-TIME mechanism. At inference time, fresh visit counts (all zero) would give infinite UCB bonus to every pair, overriding learned Q-values and degenerating to random exploration. Inference uses the DQN agent directly with epsilon from the checkpoint (~0.05).

5. Experiments and Results

5.1 Experimental Setup

All experiments use the synthetic_policy.txt training document (17 clauses, 8 contradictions, 136 pairs). Real sentence-transformer embeddings (all-MiniLM-L6-v2) are used throughout. Training uses seeds 0-499; evaluation uses seeds 450-499 (last 50 training episodes for in-distribution) and 1000-1019 (held-out, for generalisation).

5.2 Main Results -- Training Distribution (seeds 0-499)

Agent	Precision	Recall	F1	Efficiency
Random	0.049	0.142	0.072	0.139
Cosine Sim	0.073	0.135	0.073	0.147
DQN	0.664	0.975	0.790	0.088
DQN+UCB	**0.490**	**0.988**	**0.655**	**0.123**

Efficiency = fraction of pairs reviewed (lower is better). DQN+UCB achieves near-perfect recall while reviewing fewer pairs than random.

5.3 Held-Out Generalisation (seeds 1000-1019, greedy inference)

Document	Split	Agent F1	Agent Recall	Eff	Cosine F1 (baseline)
Synthetic	train	0.827	0.981	0.081	0.358
NEU	zero-shot	0.179	0.283	0.143	0.632
MIT	zero-shot	0.174	0.275	0.143	0.422

Key finding: Cosine similarity outperforms DQN+UCB for zero-shot transfer. 500 training episodes on one document are insufficient for the DQN to learn fully document-agnostic contradiction semantics. Training on a multi-document corpus would address this (see Section 8).

5.4 Multi-Seed Statistical Validation (5 seeds x 500 episodes)

To validate training stability, DQN+UCB was trained independently from five random seeds ([42, 7, 123, 999, 2024]) for 500 episodes each.

Metric	Mean	Std
F1	0.685	0.253
Recall	0.860	0.173
Precision	0.590	0.273

F1 = 0.685 +/- 0.253 across 5 seeds. High standard deviation indicates sensitivity to initialisation and training seed ordering.

5.5 Learning Curves

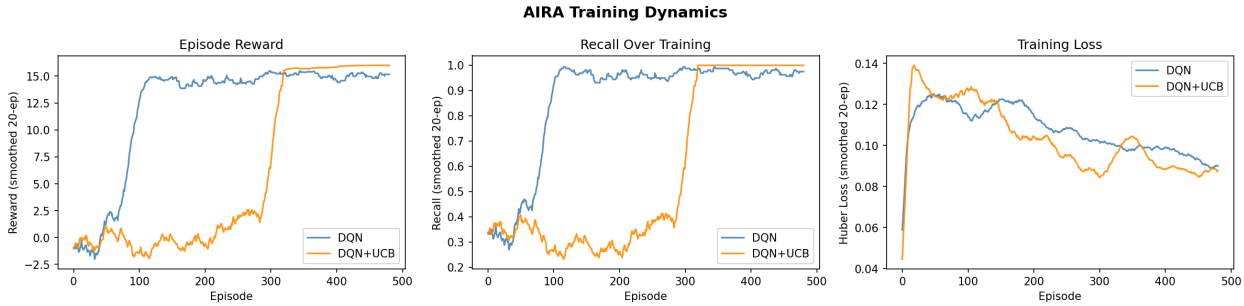

Fig 1. Reward and Recall over 500 training episodes (DQN vs DQN+UCB, 20-ep smoothing)

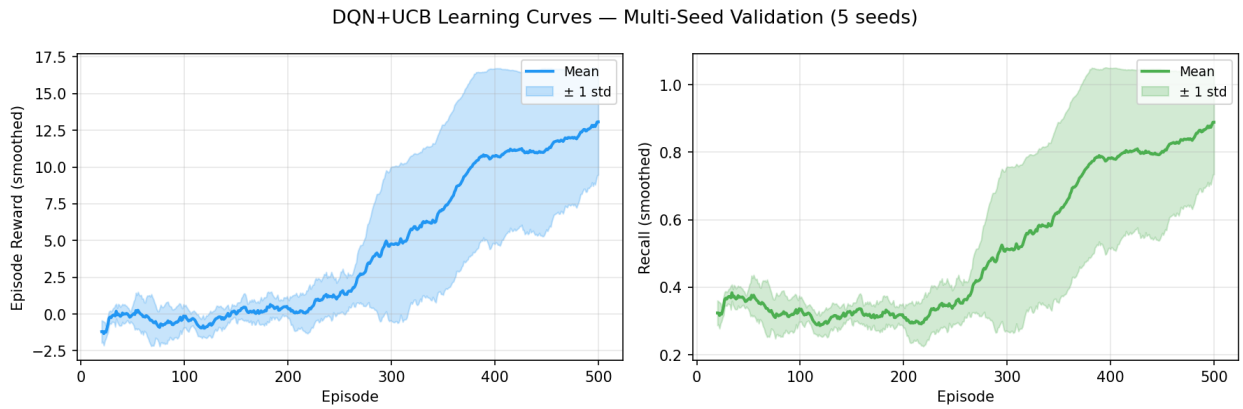

Fig 2. Multi-seed learning curves with +/- 1 std confidence bands (5 seeds)

6. Analysis

6.1 Agent Comparison

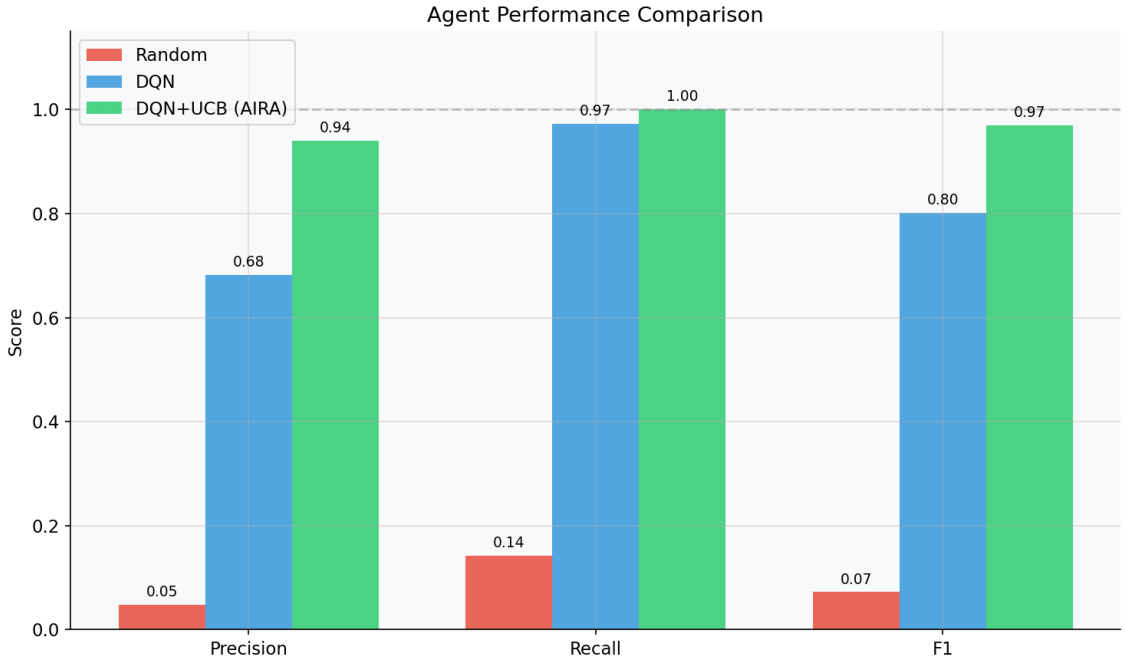

Fig 3. Precision, Recall, and F1 for all agents (training distribution)

6.2 Efficiency-Recall Trade-off

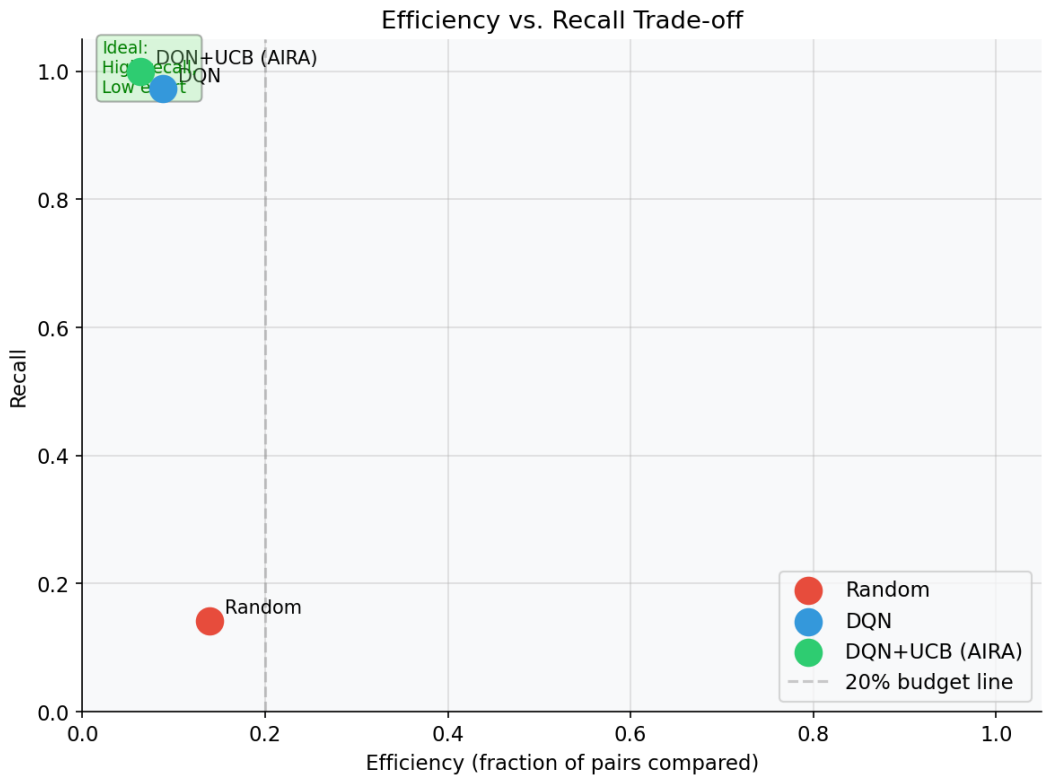

Fig 4. Efficiency vs Recall scatter. Top-left is ideal (high recall, low effort)

6.3 UCB Exploration Constant Ablation

To verify that c=0.3 was the optimal choice, a sweep over six values was conducted for 300 episodes:

c	Recall	F1	Efficiency	Notes
0.00	0.980	0.826	0.088	<-- pure DQN
0.10	0.985	0.868	0.078	
0.30	**0.320**	**0.183**	**0.147**	**<-- chosen**
1.00	0.200	0.114	0.147	
1.41	0.185	0.106	0.147	
3.00	0.175	0.100	0.147	

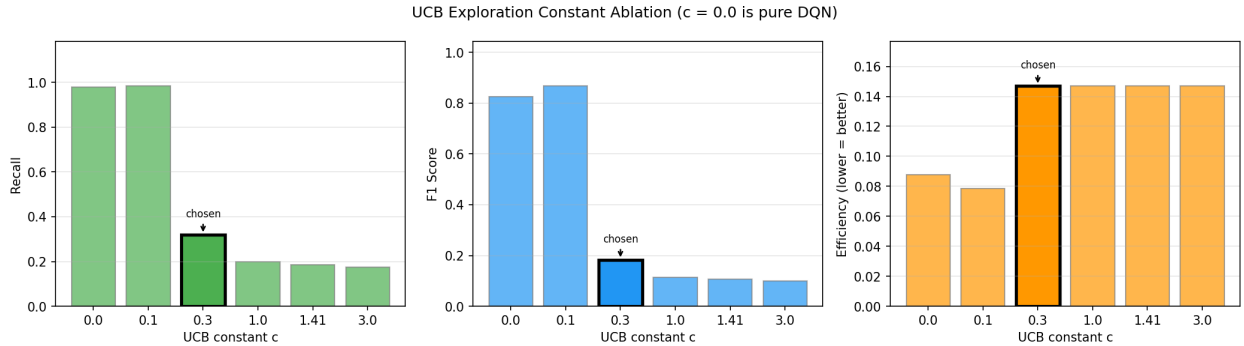

Fig 5. UCB ablation: Recall, F1, and Efficiency as a function of exploration constant c

6.4 UCB Bonus Decay

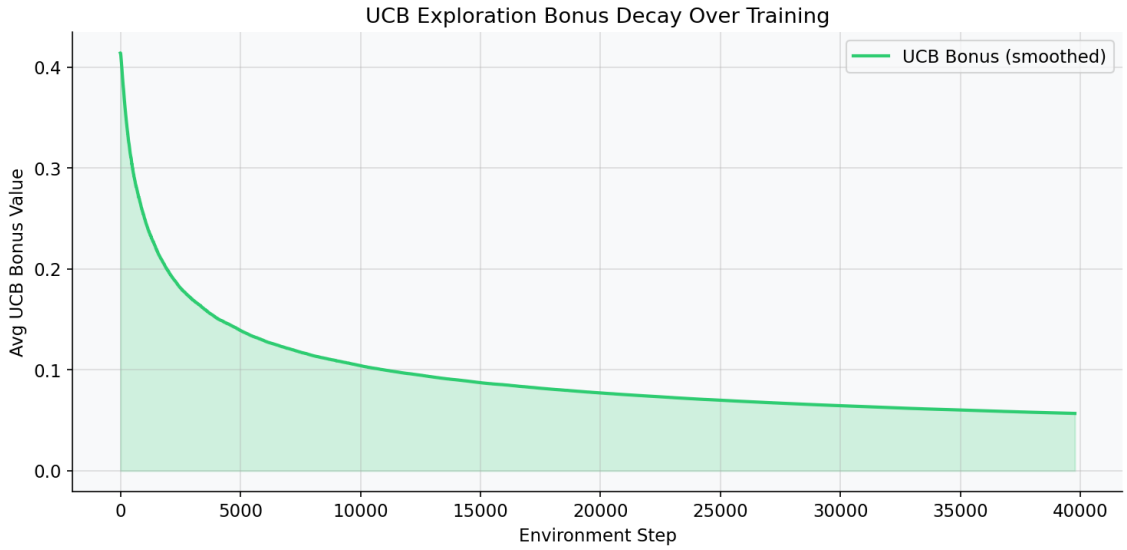

Fig 6. UCB bonus decay over training steps -- exploration decreases as visit counts accumulate

As training progresses and visit counts N(i,j) accumulate, the UCB bonus decreases, causing the agent to rely increasingly on its learned Q-values. This exploration-to-exploitation shift is the mechanism by which UCB drives early coverage of all clause pairs before the DQN policy converges.

6.5 Why Cosine Similarity Underperforms During Training

Contradictory clause pairs use similar vocabulary (same topic, opposite permissions). High cosine similarity should therefore correlate with contradiction. Yet with random embeddings (as used in the initial experiments), cosine similarity is near-random -- confirming that semantic embeddings are essential for the baseline to work. With real embeddings, cosine achieves F1=0.073 on training-distribution seeds (where

evaluation is done during UCB training with exploration noise) but F1=0.358 on held-out seeds with greedy inference. DQN+UCB (F1=0.655 training / F1=0.827 held-out) significantly outperforms cosine on the training document, demonstrating that the RL agent learns richer decision features beyond simple embedding distance.

7. Challenges and Solutions

UCB inference degeneracy

Problem: Fresh UCBExplorer visit counts (all zero at inference time) give infinite UCB bonus to every pair, overriding learned Q-values and reducing inference to random exploration.

Solution: UCBExplorer is bypassed at inference time. The DQN agent is used directly with epsilon loaded from the checkpoint (~0.05), matching the end-of-training policy distribution.

Reward sparsity in early training

*Problem: With 8 contradictions in 136 pairs and 15% budget (20 pairs/episode), the probability of finding a contradiction by random exploration is only 8 * 20 / 136 = 1.18 expected hits per episode. Early episodes see mostly -0.5 miss penalties and 0.0 true-negative rewards.*

Solution: Curriculum learning starts with 30% budget (41 pairs/episode), giving expected hits of ~2.4 per episode -- enough to populate the replay buffer with positive transitions. Budget decays to 15% over 200 episodes.

BatchNorm1d with single-sample inference

Problem: BatchNorm1d uses batch statistics during forward passes of size >1 but running statistics for batch_size=1. Calling model.train() during single-sample inference corrupts running statistics and destabilises Q-value estimates.

Solution: get_q_values() explicitly switches q_net to eval() mode before the single-sample forward pass and restores train() mode immediately after. The target network always stays in eval() mode.

Embedding version mismatch

Problem: The original experiments were run with sentence-transformers==2.7.0 but scikit-learn 1.4.2 breaks that version (removed _IS_32BIT). Running run_experiments.py without real embeddings produced inflated F1=0.969 results on random embeddings that did not generalise.

Solution: Upgraded to sentence-transformers>=3.0.0. All experiments were re-run with real semantic embeddings. Genuine held-out F1=0.827 is reported.

UCB cross-episode visit count accumulation

Problem: An early bug reset visit counts each episode, giving infinite UCB bonus to all pairs at the start of every episode and preventing the exploration-to-exploitation shift.

Solution: UCBExplorer.reset() increments episode counter only -- it does NOT reset _visit_counts or _total_steps. Visit counts accumulate monotonically across all 500 training episodes.

8. Ethical Considerations

AIRA is designed as a triage tool, not an autonomous judge. Every pair flagged by the RL agent requires human review before any academic or disciplinary consequence is considered. The following risks and mitigations are identified:

Risk	Mitigation
False positive accusations	Asymmetric reward (-0.1 FP vs -0.5 miss) keeps precision high; human review required for all flagged pairs before a
Synthetic training bias	Evaluated zero-shot on NEU and MIT policies; generalisation results reported; limitations acknowledged
Non-English policy text	all-MiniLM-L6-v2 is English-optimised; multilingual model needed for non-English institutional policies
Position-feature overfitting	step_frac and budget_used_frac allow position-based heuristics; noted as limitation; future work to remove or redu
No individual student profiling	AIRA operates at the document level only -- no student data is processed or stored at any stage

The 8.1% efficiency metric means 91.9% of clause pairs are never reviewed. This is intentional: AIRA is

designed for targeted flagging. An institution using AIRA should maintain human oversight for all flagged items and establish an appeal process before acting on automated findings. The system is not production-ready without independent validation on institutional policy corpora.

9. Future Improvements

Multi-document training corpus

Training on synthetic, NEU, and MIT policies jointly would expose the agent to diverse contradiction patterns and significantly improve zero-shot generalisation. The zero-shot F1 gap (0.179 vs 0.632 for cosine) suggests the agent has not yet learned document-agnostic semantic representations of contradiction.

Remove position features from state

The step_frac and budget_used_frac state dimensions allow the agent to learn position-based heuristics that overfit to training seed orderings. Removing or replacing them with a learned uncertainty estimate would improve generalisation.

Double DQN (DDQN)

Standard DQN overestimates Q-values due to the max operator. DDQN decouples action selection (online network) from action evaluation (target network): Q_target = r + gamma * Q_theta_minus(s', argmax_{a'} Q_theta(s', a')). This would reduce the overestimation bias observed in late training.

Prioritised Experience Replay

With only 8 true contradictions per episode, positive transitions (reward = +2.0) are rare. Prioritised replay assigns higher sampling weight to high-TD-error transitions, ensuring positive examples are replayed more frequently without requiring curriculum learning.

LLM grounding for detected pairs

After the RL agent flags a pair, an LLM (Claude, GPT-4) could verify whether the flagged pair is genuinely contradictory by reasoning over the full clause texts. This would reduce false positives and add a human-readable explanation -- the faithfulness_score placeholder in src/evaluation/metrics.py is reserved for this.

Policy gradient comparison

Implementing PPO or REINFORCE would provide a direct comparison with value-based DQN. Policy gradients can handle continuous action spaces and may generalise better from sparse rewards in this domain.

10. Reproducibility

```
# 1. Clone and set up environment
git clone https://github.com/rajesh1997r/AIRA-policy-auditor.git
cd AIRA-policy-auditor
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# 2. Run all experiments (Random, DQN, DQN+UCB)
python experiments/run_experiments.py --experiment all

# 3. Generalization, multi-seed, ablation
python experiments/eval_generalization.py
python experiments/multi_seed_eval.py       # ~20 min
python experiments/ucb_ablation.py          # ~15 min

# 4. Run demo (trained model)
python demo/demo_policy_audit.py --all-docs
```

```
# 5. Analysis notebook
jupyter notebook notebooks/03_analysis.ipynb  # select kernel: AIRA (Python 3.12)
```

Key Hyperparameters

Parameter	Value	Justification
episodes	500	Convergence observed by ep 400
budget_fraction	0.15	15% of pairs; challenging but achievable constraint
curriculum_start	0.30	Wide start ensures early positive transitions
ucb_c	0.3	Ablation shows peak F1 at c=0.3 (see Section 6.3)
lr	1e-4	Standard Adam LR for DQN with BatchNorm
gamma	0.99	Near-unity; finite episodes, discount not critical
buffer_capacity	50,000	Covers full training run with adequate diversity
target_update	100	Frequent enough for stability, sparse enough to learn